Pencil early rejection: software validation

Instrument ID: pencil-early-rejection-20261030. Isolated prospective implementation; no frozen solver, ecbench arm or experiment protocol was modified.

Implemented change

The source-bound original candidate-verification closure is retained as a test helper. The new runtime extracts that closure and checks the finalized bucket root count before computing the complement. The existing measured closure still surrounds verification. Accepted candidates retain the same geometric checks and graph path. Rejected candidates return before complement work. Editable control-flow diagram: pipeline.svg.

Checks and exact scope

Five selected deterministic software tests passed in both debug and release builds with Rust 1.99.1, two compiler jobs and one test thread. Twelve inherited tests were filtered out; some invoke solvers and were not executed. Formatting and strict Clippy on all targets passed. The debug/release runs repeat the same software checks, not independent experiments.

No solve_one invocation, rho run, cohort target, benchmark or cloud action occurred. No performance ratio is available. The independent code review found no remaining concrete defect within source/software fidelity; it does not establish scientific admission.

Fixture coverage

The complete bounded point-pair fixture inventory on one smooth Scholten curve over $\mathbb{F}_{19}$ separately reconstructs ordinary and exceptional candidate origins before comparing the old and new helpers. These are software case counts, not experimental targets:

Fixture category	Cases
Ordinary-origin candidates	612
Exceptional-only candidates	470
Exceptional accepted	214
Exceptional rejected	256
Total accepted helper comparisons	826

Polynomial-only fixtures cover rejected root counts 0, 1, 2, 4, 5. An all-lambda branch fixture uses the singular polynomial $h = x^3(x^2 + 1)$, explicitly outside smooth-curve equivalence. Phase snapshots and selected graph working-cap/pending-constraint parity are checked; this is not a general graph-state proof.

Exact counter fixture

Both arithmetic and semantic meters are enabled and bound to their moduli. In one independently derived rejected polynomial fixture, $D = 4, L = 1, T = 17, V = 16$. The old-minus-new event counts are 24 base multiplications, 30 base additions, 78 entry visits, 4 copies, 18 initializations and 27 predicate comparisons. Inversions and engine group-operation deltas are zero. These are semantic software events, not measured CPU savings.

Assumptions and limits

The field argument assumes normalized residues, a valid monic degree-two divisor, a smooth odd-characteristic quintic and bounded arithmetic ($p \leq 2^{63}$ for helper addition/subtraction). The inherited runtime API

does not enforce this narrower proof domain. Arbitrary-u64 panic, allocation, deadline and resource equivalence is not claimed. Rejected candidates intentionally skip counters and assertions.

A retained separate static proof bounds skipped complements by two per sieve and 32 over 16 walks under its smooth-quintic hypotheses. That bound supplies no speed estimate. Future measurements need prospective source/protocol/resource admission; the incomplete upper gate still blocks automatic continuation.

Provenance and development history

receipt.json binds the baseline/fork/test sources and independent review. source-and-provenance.sha256 and the dependency source manifest retain source identity. Frozen baseline SHA begins e71e9cf9; runtime fork begins 6a8041b6; tests begin 9b0ca3a2.

Failed development logs are retained. Initial exceptional-branch fixture assumptions were corrected. A later source-fidelity formatting expectation and Clippy unwrap were corrected. Independent review required explicit actual candidate-origin classification; earlier passing logs remain preserved. No empirical experiment data informed these corrections.